Aplicação de Padrões de Projeto

Marcelo Pavani Lachowski

1. Diagnóstico do projeto original

O Trabalho I segue MVC + DAO.

- Os Controllers chamam o DAO diretamente.
- O banco recusava a exclusão de um gênero com filmes, mas o usuário via a mensagem crua do PostgreSQL. A regra existia só como restrição de chave estrangeira, sem tratamento na aplicação.
- O `Filme` tinha um construtor de 6 parâmetros, com `0` e `null` passados à mão para os campos opcionais (diretor e nota).
- Excluir (ou editar) um filme não tinha volta: um engano significava perder o registro.
- Ao cadastrar um gênero na tela de Gêneros, o ComboBox da tela de Filmes não atualizava.

2. Service Layer

Identificação

Camada que concentra as regras de negócio entre o Controller e o DAO. Pacote `service`: `GeneroService`, `FilmeService`, `RegraNegocioException` e `Servicos` (instâncias compartilhadas).

Situação anterior

O Controller validava os dados e chamava o DAO direto:

```
if (nota < 0 || nota > 10) {
 Alerta.aviso("A nota deve estar entre 0 e 10.");
 return null;
}
filmeDAO.inserir(filme);
```

Solução implementada

O fluxo passou a ser Controller -> Service -> DAO. O Service valida e só então chama o DAO.
Regras assumidas pela camada:

- Filme: título e gênero obrigatórios, limites de tamanho, ano mínimo de 1888, nota entre 0 e 10 e bloqueio de título duplicado no mesmo ano (consulta no DAO).
- Gênero: nome obrigatório, limite de 50 caracteres, nome único e bloqueio de exclusão de gênero em uso, com mensagem clara contando os filmes vinculados.

```
// FilmeService
public void inserir(Filme filme) throws SQLException, RegraNegocioException {
validar(filme); // regras de negócio
 dao.inserir(filme); // persistência
}

// GeneroService
public void excluir(Genero genero) throws SQLException, RegraNegocioException {
int quantidade = filmeDAO.contarPorGenero(genero.getId());
 if (quantidade > 0) {
 throw new RegraNegocioException("Não é possível excluir: existem "  +
quantidade + " filme(s) com este gênero.");
 }
 generoDAO.excluir(genero.getId());
```

```
 notificarObservers();
}

// FilmeController: só lê a tela e mostra o resultado
try {
 filmeService.inserir(filme);
 limpar();
 carregarFilmes();
} catch (RegraNegocioException e) {
 Alerta.aviso(e.getMessage()); // regra violada
} catch (SQLException e) {
 Alerta.erro("Erro ao inserir filme: " + e.getMessage()); // falha técnica }
```

Justificativa e verificação

O Controller deixou de conhecer regras e banco, o Service pode ser testado sem interface. A camada tem responsabilidade própria, não apenas repassa chamadas.

- Nenhum Controller importa mais as classes do pacote `dao`: todos usam `Servicos.FILME` e `Servicos.GENERO`.
- Teste de lógica com DAOs simulados: nota 11, título vazio, ano 1500 e título duplicado foram recusados com a mensagem correta; excluir gênero com 2 filmes foi recusado e, sem filmes, foi aceito.
- Na aplicação: tentar excluir um gênero com filmes mostra o aviso "existem N filme(s) com este gênero" em vez do erro do PostgreSQL.

3. Observer

Identificação

Faz as telas reagirem a mudanças nos gêneros. Papéis: `GeneroService` (Subject), `GeneroObserver` (interface, pacote `observer`) e `FilmeController` (Observer concreto).

Situação anterior

O ComboBox de gêneros da tela de Filmes era preenchido uma única vez, no `initialize()`. Um gênero criado, renomeado ou excluído na outra tela só aparecia ao fechar e reabrir a tela de Filmes.

Solução implementada

```
// GeneroObserver
public interface GeneroObserver { void generosAlterados(); }

// GeneroService
private final List<GeneroObserver> observers = new ArrayList<>();
public void adicionarObserver(GeneroObserver o) { observers.add(o); }
public void removerObserver(GeneroObserver o) { observers.remove(o); }
private void notificarObservers() {
 for (GeneroObserver o : new ArrayList<>(observers)) o.generosAlterados(); }
// inserir, atualizar e excluir chamam notificarObservers() após gravar

// FilmeController
generoService.adicionarObserver(this); // no initialize()
@Override
public void generosAlterados() {
 carregarGeneros(); // recarrega o ComboBox
 carregarFilmes(); // nome do gênero pode ter mudado }
```

Ao fechar a janela de Filmes, o controller se remove da lista de observers (`setOnHidden`), evitando atualizar uma tela que não existe mais.

Justificativa e verificação

- Resolve um defeito real do Trabalho I e desacopla as telas: o `GeneroController` não conhece o `FilmeController`; só o Service sabe quem está inscrito.
- Teste de lógica: com um observer inscrito, inserir um gênero gerou exatamente 1 aviso, depois de removido o observer, uma nova inserção não gerou aviso.
- Na aplicação: com as duas telas abertas, cadastrar um gênero novo faz ele aparecer imediatamente no ComboBox da tela de Filmes.

4. Builder

Identificação

Monta objetos `Filme` de forma legível. Classes: `FilmeBuilder` (Builder) e `Filme` (produto), no pacote `model`.

Situação anterior

```
return new Filme(id, titulo, diretor, ano, nota, genero);
// 6 parâmetros posicionais; diretor e nota são opcionais (null)
// e o id é 0 quando o filme ainda não existe no banco
```

Solução implementada

O `FilmeBuilder` tem um método por campo, devolvendo `this`, e o `build()` cria o filme. O construtor de `Filme` passou a ser visível só dentro do pacote, então **a única forma de criar um Filme fora do pacote model é pelo Builder**.

```
// FilmeController.lerFormulario()
return new FilmeBuilder()
 .id(id)
 .titulo(txtTitulo.getText().trim())
 .diretor(txtDiretor.getText().trim())
 .ano(ano)
 .nota(nota)
 .genero(cmbGenero.getValue())
 .build();

// FilmeDAO.listar(): mesmo Builder ao ler do ResultSet
lista.add(new FilmeBuilder().id(rs.getInt("id")).titulo(rs.getString("titulo"))
.diretor(rs.getString("diretor")).ano(rs.getInt("ano"))
 .nota(nota).genero(genero).build());
```

Justificativa e verificação

- Cada valor passado fica identificado pelo nome do método, e os campos opcionais podem ser simplesmente omitidos.
- O Builder é usado nos dois pontos onde filmes nascem: ao ler o formulário (Controller) e ao ler o banco (DAO).
- Verificação: o compilador recusa `new Filme(...)` fora do pacote ("Filme(...) is not public in Filme"), e o teste criou filmes sem diretor e sem nota (ficam null).

5. Command

Identificação

Transforma cada ação sobre filmes em um objeto que sabe se executar e se desfazer, permitindo o botão "Desfazer". Pacote command: Comando (interface Command), InserirFilmeCommand, AtualizarFilmeCommand e ExcluirFilmeCommand (comandos concretos), HistoricoComandos (Invoker), FilmeService (Receiver) e FilmeController (Client).

Situação anterior

As operações eram chamadas direto no service e não deixavam rastro. Depois de excluir ou editar um filme por engano, o usuário não tinha como voltar atrás.

Solução implementada

```
// command/Comando.java
public interface Comando {
 void executar() throws SQLException, RegraNegocioException;
 void desfazer() throws SQLException, RegraNegocioException;
 String descricao();
}

// ExcluirFilmeCommand: desfazer = restaurar o filme com o mesmo id
public void executar() throws SQLException { service.excluir(filme.getId()); }
public void desfazer() throws SQLException, RegraNegocioException {
 service.restaurar(filme);
}

// HistoricoComandos
public void executar(Comando comando) throws SQLException, RegraNegocioException {
comando.executar(); // só entra no histórico se executou sem erro
pilha.push(comando);
}
public void desfazer() throws SQLException, RegraNegocioException {
 if (pilha.isEmpty()) return;
 pilha.pop().desfazer();
}

// FilmeController
historico.executar(new ExcluirFilmeCommand(filmeService, selecionado));
```

Extensões justificadas, necessárias para o padrão: (1) botão "Desfazer" na tela de Filmes, que mostra a ação a ser revertida; (2) FilmeDAO.inserir passou a devolver o id gerado pelo banco, para o comando de inserção saber o que remover; (3) FilmeDAO.restaurar reinsere o filme mantendo o id original, para que comandos anteriores do histórico continuem válidos.

Justificativa e verificação

- Cada ação vira um objeto, e o histórico não conhece os detalhes de nenhuma. Tornar outra ação desfazível exige só uma nova classe de comando.
- Teste de lógica com DAO em memória: inserir A e B, atualizar a nota de A, excluir A e desfazer quatro vezes devolveu exatamente cada estado anterior (A voltou com o mesmo id e a nota correta). Desfazer com histórico vazio não quebra, e um comando recusado pelas regras do Service não entra no histórico.